

April 10, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, getcontext # Besonders für sig.
↳ Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
```

```

import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "213"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\213

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\Code\221\Diagramme

Runden signifikanter Stellen

```

[3]: def round_to_sigs(val, errVal, einheiten):
    """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↪ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↪ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↪ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert
    """

```

```

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
    """

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳ signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳ erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳ hinzu
        exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_CEILING) *
↳ scale # mit scale wird das Komma so verschoben, dass alle
↳ signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳ weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳ * scale

qty = f"\qty{{{val_round}}({err_round})}{{{einheiten}}}"
num = f"\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳ excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳ gewollt sind, die 10~e Präfixe beinhalten)

```

Gausssche Fehlerfortpflanzung

```

[4]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamaters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

    for variable in variables:
        Delta = sp.Symbol(f"Delta_{variable}")
        partial = sp.diff(function, variable) # Die Funktion
    ↪wird nach der fehlerbehafteten Variable abgeleitet
        term=sp.UnevaluatedExpr(partial*Delta)**2
        error = error + ((term)) # Fehler werden
    ↪quadratisch aufsummiert
        errProneParamaters.append((variable,Delta))

    absolute_err=sp.simplify(sp.sqrt(error),rational = True)
    relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

```

```

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[5]: def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
    ↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

        exp = int(math.floor(math.log10(float(abweichung))))
        if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
        ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
        ↪Nachkommastellenposition hinzu
            exp -= 1
        scale = Decimal(10) ** exp
        abweichung_round = (abweichung / scale).quantize(Decimal('1'),
        ↪rounding=ROUND_CEILING) * scale
        res = f"\num{{{abweichung_round}}}"

    return float(abweichung_round),res

```

```

[6]: gff("h_1/(h_1-h_3)","h_1,h_3")

```

Funktion:

$$\frac{h_1}{h_1 - h_3}$$

$\frac{h_1}{h_1 - h_3}$

Absoluter Fehler:

$$\sqrt{\frac{1}{(h_1 - h_3)^4} (\Delta_{h_1}^2 h_3^2 + \Delta_{h_3}^2 h_1^2)}$$

$$\sqrt{\frac{\Delta_{h_1}^2 h_3^2 + \Delta_{h_3}^2 h_1^2}{h_1^2 (h_1 - h_3)^2}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{h_1}^2 h_3^2 + \Delta_{h_3}^2 h_1^2}{h_1^2 (h_1 - h_3)^2}}$$

$$\sqrt{\frac{\Delta_{h_1}^2 h_3^2 + \Delta_{h_3}^2 h_1^2}{h_1^2 (h_1 - h_3)^2}}$$

```
[6]: (h_1/(h_1 - h_3),
      sqrt((Delta_h_1**2*h_3**2 + Delta_h_3**2*h_1**2)/(h_1 - h_3)**4),
      sqrt((Delta_h_1**2*h_3**2 + Delta_h_3**2*h_1**2)/(h_1**2*(h_1 - h_3)**2)),
      [(h_1, Delta_h_1), (h_3, Delta_h_3)])
```

```
[7]: h_1=np.array([7.85,6.25,4.50,4.00])
      h_3=np.array([2.20,1.65,1.20,1.10])
      Delta_h_1=Delta_h_3=round_to_sigs(h_1[0],np.sqrt(2)*0.5*1e-1,r"m")[1]
      kappa_CD=(h_1/(h_1 - h_3))
      Delta_kappa_CD=np.sqrt((Delta_h_1**2*h_3**2 + Delta_h_3**2*h_1**2)/(h_1 -
      ↪h_3)**4)
      kappa_CD,Delta_kappa_CD,latexkappa,_ = v_round(kappa_CD, Delta_kappa_CD,r"\m")
      print(kappa_CD)
      print(Delta_kappa_CD)
      bar_kappa_CD=np.sum(kappa_CD/Delta_kappa_CD**2)/np.sum(1/Delta_kappa_CD**2)
      bar_Delta_kappa_CD=np.sqrt(1/np.sum(1/Delta_kappa_CD**2))
      for i in range(len(h_1)):
          ↪
          ↪print(f"\num{{{h_1[i]}}({Delta_h_1})}&\num{{{h_3[i]}}({Delta_h_1})}&{latexkappa[i]}\\\\")
      print(round_to_sigs(bar_kappa_CD,bar_Delta_kappa_CD,r"")[2])
```

```
[1.39  1.359 1.37  1.38 ]
[0.021 0.025 0.04  0.04 ]
\num{7.85(0.08)}&\num{2.2(0.08)}&\num{1.390(0.021)}\\
\num{6.25(0.08)}&\num{1.65(0.08)}&\num{1.359(0.025)}\\
\num{4.5(0.08)}&\num{1.2(0.08)}&\num{1.37(0.04)}\\
\num{4.0(0.08)}&\num{1.1(0.08)}&\num{1.38(0.04)}\\
\num{1.377(0.014)}
```

```
[8]: gff("p_0+m*g/(pi*r**2)","p_0,m,g,r")
```

Funktion:

$$\frac{gm}{\pi r^2} + p_0$$

$$\frac{g m}{\pi r^2} + p_0$$

Absoluter Fehler:

$$\frac{1}{\pi} \sqrt{\frac{1}{r^6} (\pi^2 \Delta_{p_0}^2 r^6 + 4 \Delta_r^2 g^2 m^2 + r^2 (\Delta_g^2 m^2 + \Delta_m^2 g^2))}$$

$$\frac{\sqrt{\frac{\pi^2 \Delta_{p_0}^2 r^6 + 4 \Delta_r^2 g^2 m^2 + \Delta_m^2 g^2 r^2}{\pi^2 \Delta_{p_0}^2 r^6 + 4 \Delta_r^2 g^2 m^2 + \Delta_m^2 g^2 r^2}}}{\pi}$$

Relativer Fehler:

$$\sqrt{\frac{1}{r^2 (g m + \pi p_0 r^2)^2} (\pi^2 \Delta_{p_0}^2 r^6 + 4 \Delta_r^2 g^2 m^2 + r^2 (\Delta_g^2 m^2 + \Delta_m^2 g^2))}$$

$$\sqrt{\frac{\pi^2 \Delta_{p_0}^2 r^6 + 4 \Delta_r^2 g^2 m^2 + \Delta_m^2 g^2 r^2}{r^2 (g m + \pi p_0 r^2)^2}}$$

[8]: (g*m/(pi*r**2) + p_0,
sqrt((pi**2*Delta_p_0**2*r**6 + 4*Delta_r**2*g**2*m**2 + r**2*(Delta_g**2*m**2 + Delta_m**2*g**2))/r**6)/pi,
sqrt((pi**2*Delta_p_0**2*r**6 + 4*Delta_r**2*g**2*m**2 + r**2*(Delta_g**2*m**2 + Delta_m**2*g**2))/(r**2*(g*m + pi*p_0*r**2)**2)),
[(p_0, Delta_p_0), (m, Delta_m), (g, Delta_g), (r, Delta_r)])

[11]: gff("4*m*V/(r**4*T**2*p)", "m,V,r,T,p")

Funktion:

$$\frac{4Vm}{T^2 p r^4}$$

$$\frac{4 V m}{T^2 p r^4}$$

Absoluter Fehler:

$$4 \sqrt{\frac{1}{T^6 p^4 r^{10}} (4 \Delta_T^2 V^2 m^2 p^2 r^2 + \Delta_p^2 T^2 V^2 m^2 r^2 + 16 \Delta_r^2 T^2 V^2 m^2 p^2 + T^2 p^2 r^2 (\Delta_V^2 m^2 + \Delta_m^2 V^2))}$$

$$4 \sqrt{\frac{\Delta_T^2 V^2 m^2 p^2 r^2 + \Delta_p^2 T^2 V^2 m^2 r^2 + 16 \Delta_r^2 T^2 V^2 m^2 p^2 + T^2 p^2 r^2 (\Delta_V^2 m^2 + \Delta_m^2 V^2)}{T^6 p^4 r^{10}}}$$

Relativer Fehler:

$$\sqrt{\frac{4 \Delta_T^2}{T^2} + \frac{\Delta_V^2}{V^2} + \frac{\Delta_m^2}{m^2} + \frac{\Delta_p^2}{p^2} + \frac{16 \Delta_r^2}{r^2}}$$

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_V^2}{V^2} + \frac{\Delta_m^2}{m^2} + \frac{\Delta_p^2}{p^2} + \frac{16 \Delta_r^2}{r^2}}$$

```
[11]: (4*V*m/(T**2*p*r**4),
      4*sqrt((4*Delta_T**2*V**2*m**2*p**2*r**2 + Delta_p**2*T**2*V**2*m**2*r**2 +
      16*Delta_r**2*T**2*V**2*m**2*p**2 + T**2*p**2*r**2*(Delta_V**2*m**2 +
      Delta_m**2*V**2))/(T**6*p**4*r**10)),
      sqrt(4*Delta_T**2/T**2 + Delta_V**2/V**2 + Delta_m**2/m**2 + Delta_p**2/p**2 +
      16*Delta_r**2/r**2),
      [(m, Delta_m), (V, Delta_V), (r, Delta_r), (T, Delta_T), (p, Delta_p)])
```

```
[10]: print(sigma_abweichung(1.377,1.401,0.014,0)[1])
      print(sigma_abweichung(1.357,1.401,0.018,0)[1])
      print(sigma_abweichung(1.603,1.648,0.023,0)[1])
      print(sigma_abweichung(1.377,1.357,0.023,0.14)[1])
```

\num{1.8}

\num{2.5}

\num{2.0}

\num{0.15}